CSC3100 Final Review

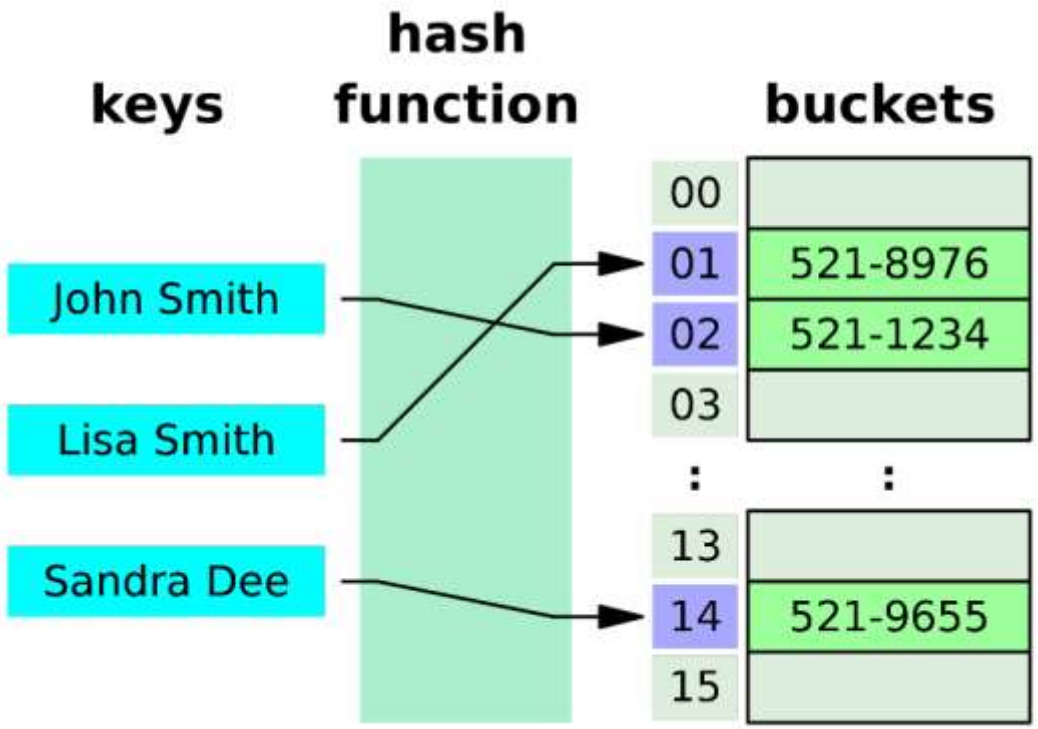
CUHK-Shenzhen Programming Contest Team

Runyuan He & Xinyan Zhou

Review of core course topics

- (1) **Hash table**: chaining, linear probing, double hashing, definition of alpha, time complexity and its proof.
- (2) **Heap**: insert, delete, building a heap in linear time
- (3) **Stack**: how to do DFS in a stack, DFS tree, compare it and implementing BFS with a queue. (first in last out, first in first out)
- (4) **Tree**: definition; AVL Tree: rotate
- (5) **Graph**: comparison of linked list and linked table (how to use, complexity), Floyd(why it's DP), shortest path, MST

Hash table



In C++ : `map`
In Python : `dict()`

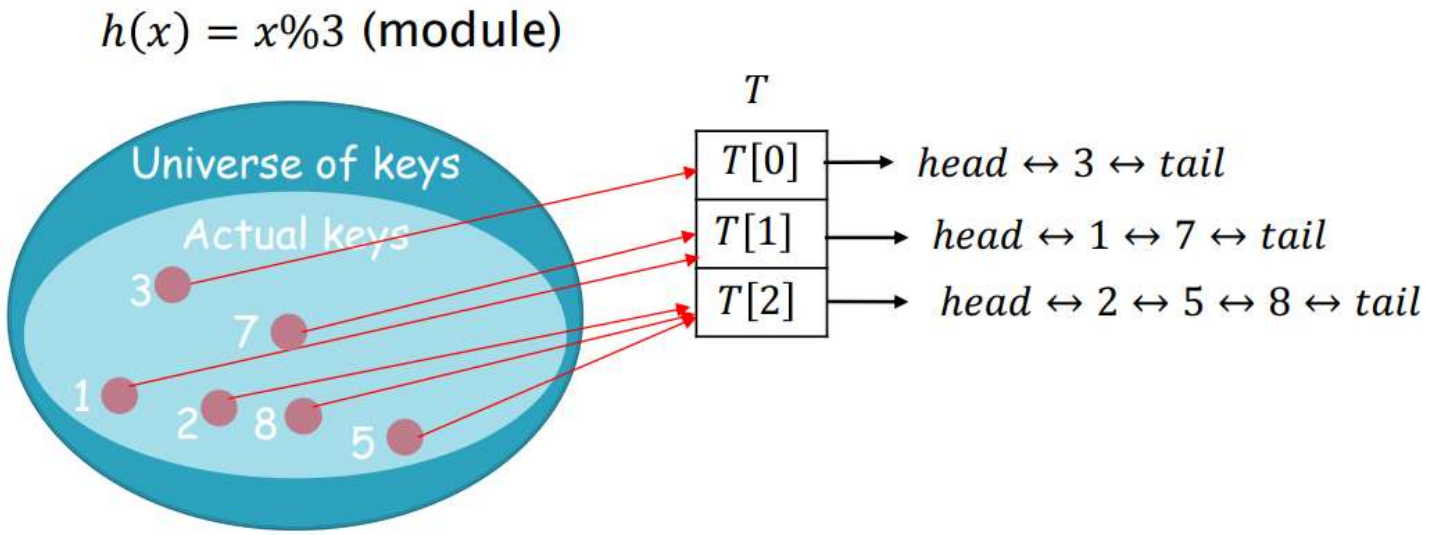
The main problem is to solve collision.

Collision: two keys hash to the same slot

Load factor: $\alpha = \frac{n}{m}$, where n is number of elements in hash table, m is the size of hash table.

Hash table

- **Chaining:** we place all elements that hash to the same slot into the same linked list
- **Time Complexity:** search/insertion/deletion takes $O(\alpha + 1)$ time

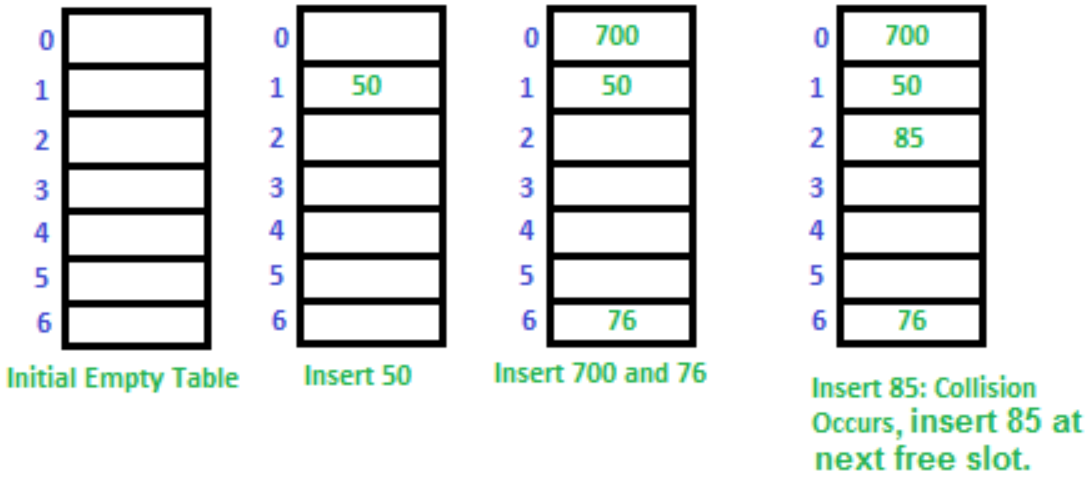


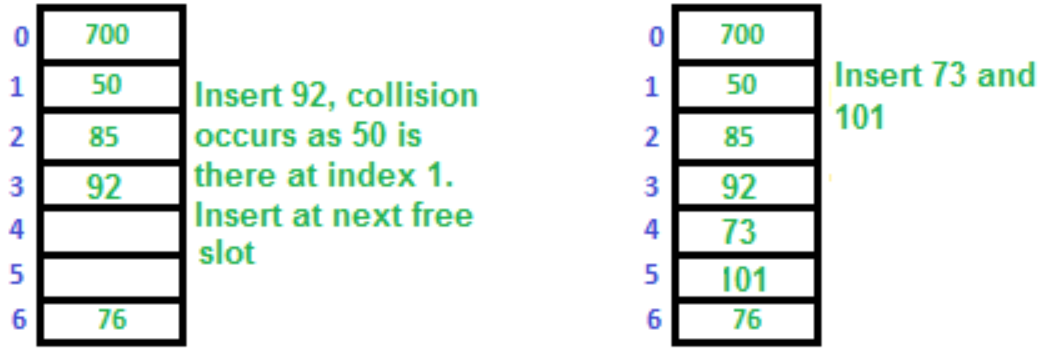
Hash table

- **Open-addressing:** All elements are stored in the hash table
  - Unlike chaining, no elements are stored **outside** the table
  - The hash table may "**fill up**" such no insertion can be made
- **Insertion:** we examine, or probe, the hash table until an empty slot is found to put the key
  - Linear probing
  - Double Hashing
- **Deletion:** When deleting an element, we cannot simply set the slot into NULL because it can affect the process to find an element later.
- **Time Complexity:** $O\left(\frac{\alpha}{1-\alpha}\right)$

Hash table

Open Addressing (linear probing)
Insert 50, 700, 76, 85, 92, 73, 101





Hash table

- Linear Probing

For insertion, we probe $h(k), h(k)+1, h(k)+2, \cdots, h(k)+(m-1)$ one by one until we find an empty slot, and insert the record to this slot

- Formally we probe $h(k,i) = (h(k)+i)\%m$ from $i = 0$ to $i = m-1$ until we find an empty slot to insert

When searching for a record with a certain key,

- Compute $h(k)$
- Examine the hash table buckets in order $T[h(k,i)]$ for $0 \le i \le m-1$ until one of the following happens:
  - $T[h(k,i)]$ has the record with key equal to k
  - $T[h(k,i)]$ is empty, then no record contains key k in the hash table

In one word:

- If the slot is full, then go to the next slot, until an empty slot is found.

Shortcoming: Long sequence of occupied slots, which degrades the query efficiency

Proof of time complexity

- Given load factor α, if we randomly choose a slot in the hash table, the probability of the slot is empty is $1 - \alpha$.
- We will not stop until we find an empty slot to insert. The process can be viewed as a geometric distribution. $E(x) = (1 - p)/p$, where $p = 1 - \alpha$.
- Substitute in p will give us $O(\frac{\alpha}{1-\alpha})$.

Hash table

Double hashing

- We have an additional hash function $h' > 0$
- Insertion: we probe $h(k,i) = \big(h(k) + i \cdot h'(k)\big)\%m$ one by one for i from 0 to $m-1$ until an empty slot is found
- Search: we search $h(k,i)$ for i from 0 to $m-1$ until one of the following happens:
  - $T[h(k,i)]$ has the record with key equal to k
  - $T[h(k,i)]$ is empty, then no record contains key k in the hash table

Hash table

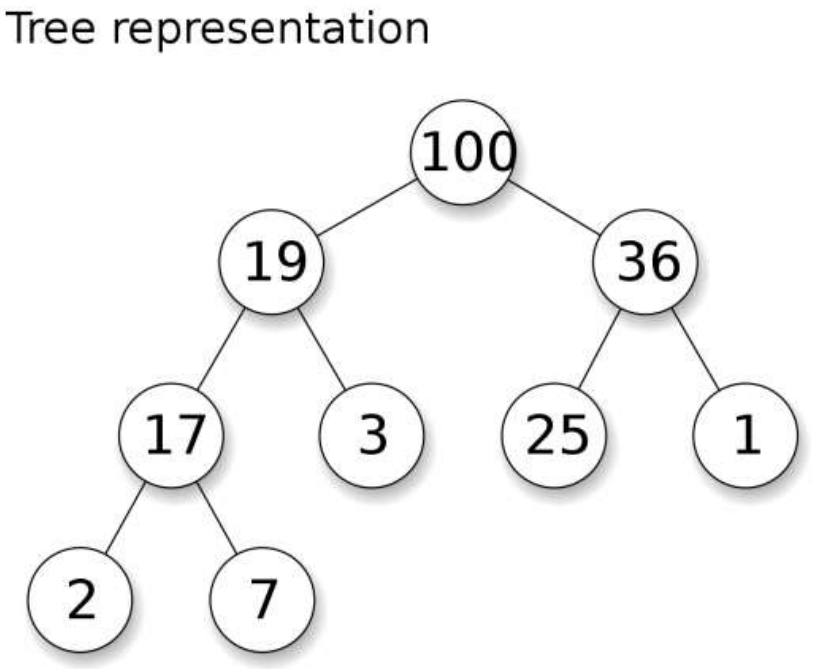
- Pros of chaining
  - Less sensitive to hash functions and load factors (α can be larger than 1), while open addressing requires to avoid long probes, and its load factor $\alpha < 1$
  - Support deletion, while open addressing is difficult to support deletion (why?)

- Pros of open addressing
  - Usually much faster than chaining

Heap

- **Heap property**
- The key stored in each node is either greater than or equal to (≥) or less than or equal to (≤) the keys in the node's **children**

- **Shape property**
- The binary tree is completely filled, except at the bottom level, which is filled from left to right



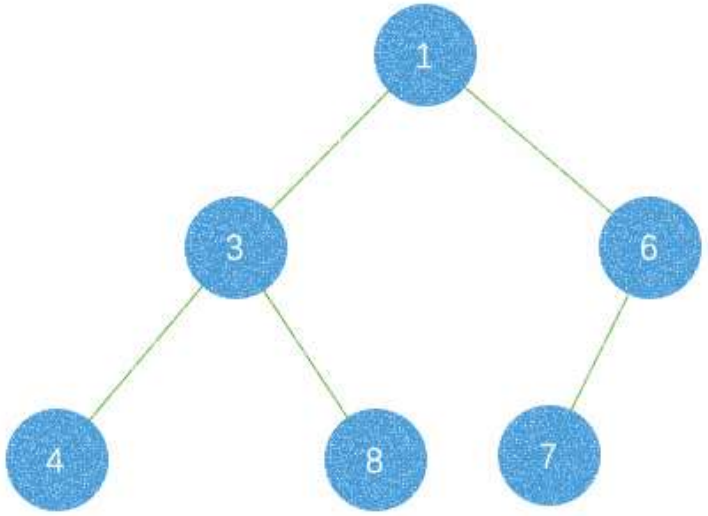
Pseudo code (build heap)

```
Heapify(arr, n, i)
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2
    if left < n and arr[left] > arr[largest]
        largest = left
    if right < n and arr[right] > arr[largest]
        largest = right
    if largest != i
        swap(arr[i], arr[largest])
        heapify(arr, n, largest)
```

```
BuildHeap(arr)
    n = arr.size();
    for i = n / 2 - 1 downto 0
        Heapify(arr, n, i)
```

Heap

- To **insert** an element X,
- 1. Create a hole in the next available location
- 2. If X can be placed in the hole without violating heap order, insertion is done
- 3. Otherwise slide the element that is in the hole's parent node into the hole,
- 4. Continue this process until X can be placed in the hole (a percolating up
- process)

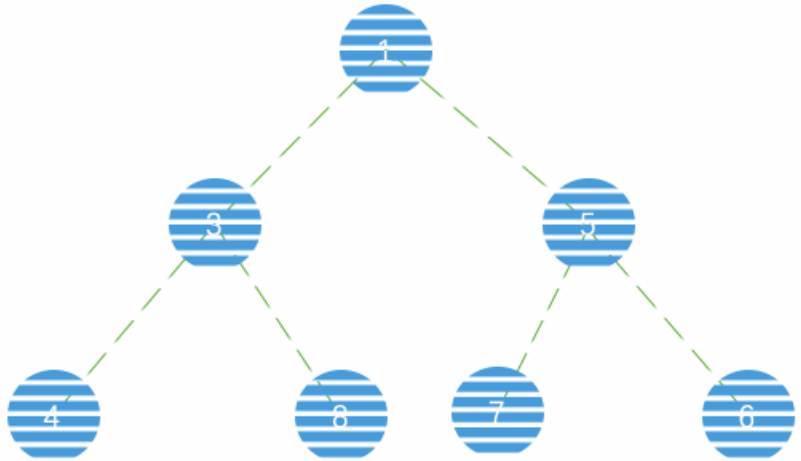


Pseudo code (build heap)

```
HeapifyUp(arr, index)
    parent = (index - 1) / 2;
    while index > 0 and arr[index] > arr[parent]
        swap(arr[index], arr[parent]);
        index = parent;
        parent = (index - 1) / 2;
Insert(val)
    if heapSize == MAX_SIZE
        return
    heap[n] = val // insert new element to the end
    n++
    HeapifyUp(heapSize – 1) // adjust the heap until stable
```

Heap

- To **Delete** an element X,
- 1. The element at the root (position 1) is to be removed, and a hole is created
- 2. Fill the root with the last node X
- 3. Percolate X down (switch X with the smaller child) until the heap order property is satisfied



Pseudo code (build heap)

```
DeleteHeapTop(arr, n)
    if (n <= 0)
        return
    swap(arr[0], arr[n - 1])
    n--
    Heapify(arr, n, 0)
```

Heap

- Complexity: $O(\log n)$ for both insertion and deletion

Complexity of buildHeap?

- Analysis
  - percolateDown for n/2 keys
  - Each key takes up to O(logn) cost
  - Thus, the total cost of BuildHeap is O(nlogn)

Is this upper bound tight?

- Notice
  - At most n/4 percolate down 1 level
    at most n/8 percolate down 2 levels
    at most n/16 percolate down 3 levels
    …

$$1\frac{n}{4}+2\frac{n}{8}+3\frac{n}{16}+\ldots=\sum_{i=1}^{\log n} i\frac{n}{2^{i+1}}=$$

$$\frac{n}{2}\sum_{i=1}^{\log n}\frac{i}{2^i}\approx\frac{n}{2}(2)=n$$

Conclusion: O(n)

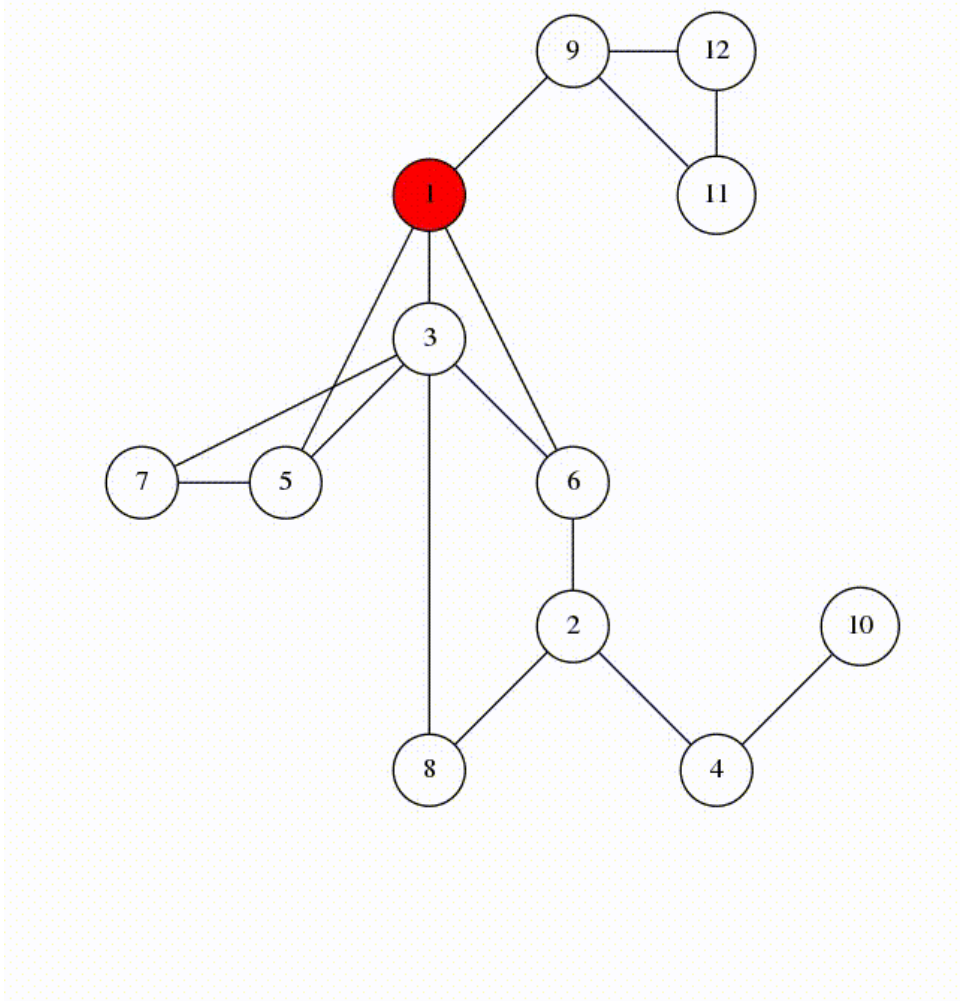
Stack

- How to do **DFS** in a stack?
- 1. Pop the top element **x** from the stack.
- 2. Use **x** to explore new states
- 3. Push new states into the stack.

```
void dfs(int start) {
    stack.push(start);
    vis[start] = true;
    while(!stack.empty()) {
        int x = stack.top();
        stack.pop();
        for(v : adjlist[x]) {
            if (!vis[v]){
                stack.push(v);
                vis[v] = true;
            }
        }
    }
}
```

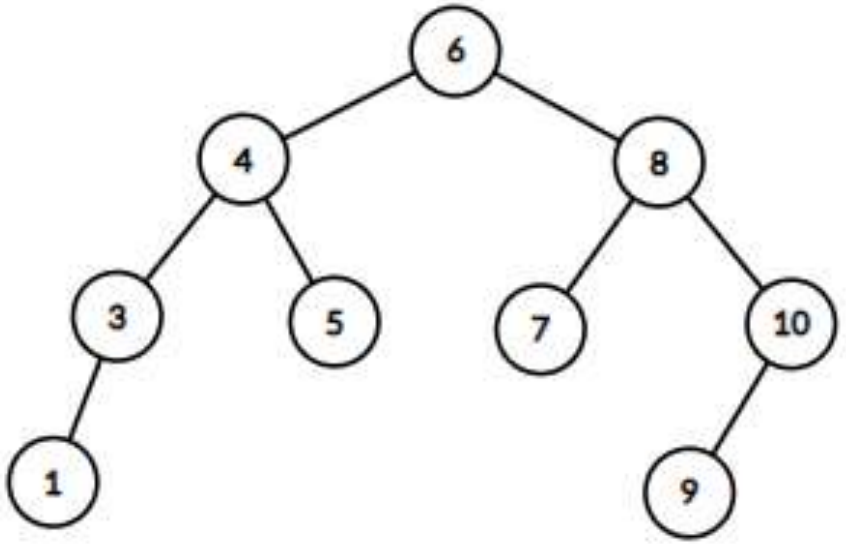
Stack

- **DFS Tree**



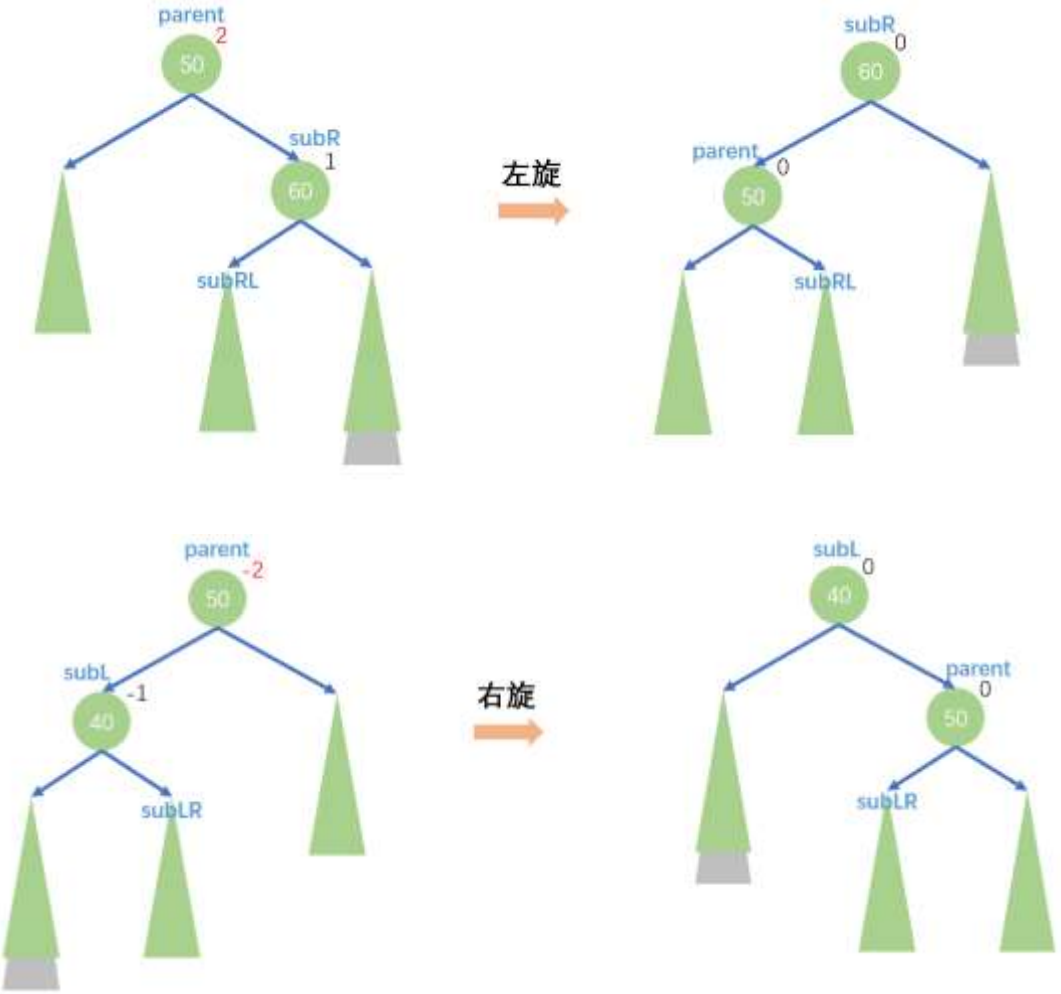
Tree & AVL Tree

- **Tree** : A connected graph with n nodes and n – 1 edges.
- **AVL Tree** : An AVL tree is a self-balancing BST
- **Balancing**: For every node in the tree, the height of the left subtree differs from the height of the right subtree by at most 1



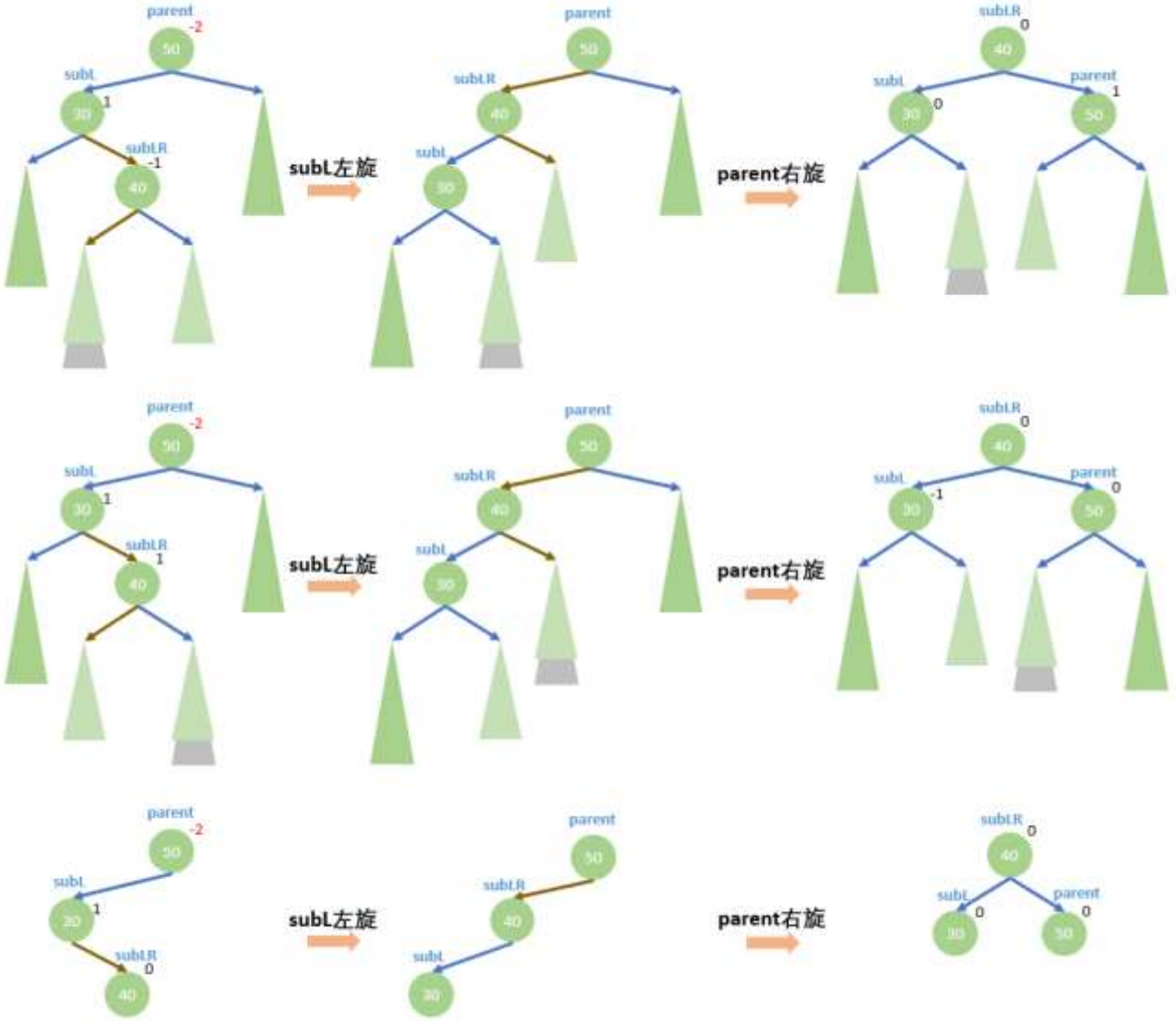
Tree & AVL Tree

- Rotate



Tree & AVL Tree

- Rotate



Pseudo code (Rotation)

```
Left-Rotate(x):
    y = x.right
    x.right = y.left
    if y.left != NULL:
        y.left.parent = x
        y.parent = x.parent
    if x.parent == NULL:
        root = y
    else if x == x.parent.left:
        x.parent.left = y
    else:
        x.parent.right = y
    y.left = x
    x.parent = y
```

```
Right-Rotate(x):
    y = x.left
    x.left = y.right
    if y.right != NULL:
        y.right.parent = x
    y.parent = x.parent
    if x.parent == NULL:
        root = y
    else if x == x.parent.right:
        x.parent.right = y
    else:
        x.parent.left = y
    y.right = x
    x.parent = y
```

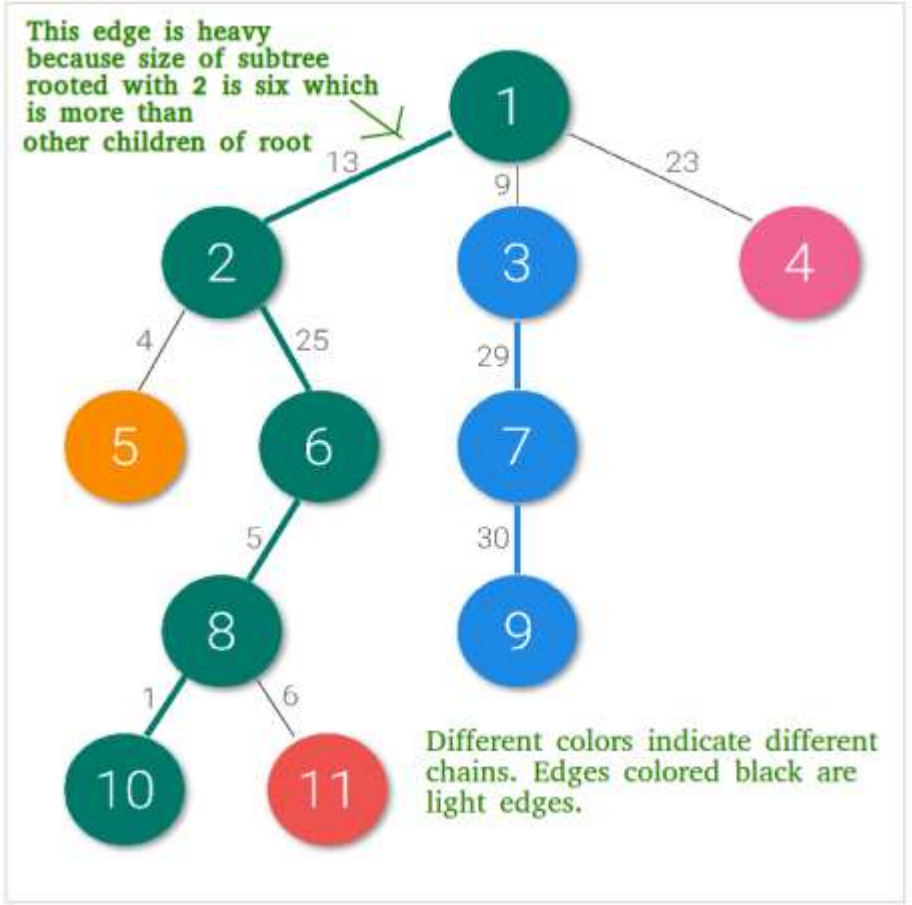
Pseudo code (double rotation)

```
Left-Right-Rotate(x):
    Left-Rotate(x.left)
    Right-Rotate(x)

Right-Left-Rotate(x):
    Right-Rotate(x.right)
    Left-Rotate(x)
```

Tree & AVL Tree

- **Sample question 1** Given the algorithm of heavy-light decomposition:
  - Define the heavy child of each vertex as the child with the largest subtree size.
  - The tree is split into top-to-bottom chains with each vertex pointing to its heavy child.
- Prove any path on the tree will have at most $O(\log(n))$ heavy chains.
- Given the following arrays, design an algorithm to locate all heavy chains between any 2 vertices in $O(\log(n))$ time.
  - `top[u]`: the top vertex of the chain of vertex u
  - `par[u]`: u's parent on the tree
  - `dep[u]`: u's depth on the tree



Tree & AVL Tree

- Any path from one vertex to the root will have at most $\log n$ heavy chains.
  - Define `size[u]` as the size of subtree rooted at u
  - Consider any vertex u on the path that is the head of some heavy chains.
  - The heavy child of `par[u] = v ≠ u, size[v] ≥ size[u]`
  - `size[par[u]] ≥ size[u] + size[v] ≥ 2size[u]`
  - List all u_i from bottom to top:
    - $u_1, u_2, \cdots, u_k$, k is the number of heavy chains, $u_k = 1$ (the root node)
  - `size[`u_i`] ≤` $\frac{1}{2}$ `size[par[`u_i`]] ≤ size[`u_{i+1}`]`
  - `size[`u_1`] ≥` $2^{k-1} \Rightarrow k \sim O(\log n)$

Tree & AVL Tree

```
int locate(int u, int v) {
  while (top[u] != top[v]) {
    if (dep[top[u]] > dep[top[v]])
      u = fa[top[u]];
    else
      v = fa[top[v]];
  }
  return dep[u] > dep[v] ? v : u;
}
```

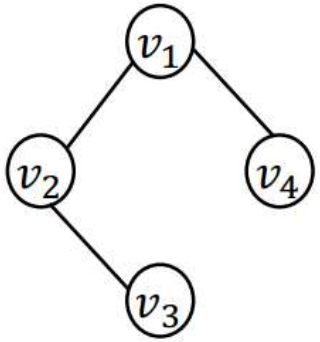
Tree & AVL Tree

- **Sample question 2**
- Given a tree with n vertices. Each vertice has weight vi. Find a subset with the maximal total weight of vertices on the tree, such that there's no edge on the tree between each two vertices in the subset.
- Expected Time & space complexity: $O(n)$

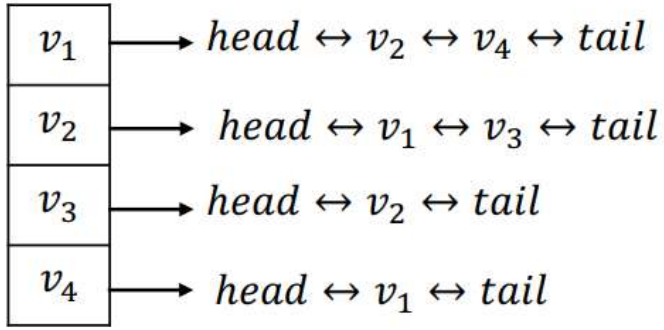
```
int dp[100001][2];
// dp[i][j] denotes choose vertex i if j=1 and not choose vertex i if j=0
void dfs(int x){
        for(int v : adj[x]){
                dfs(v);
                dp[x][1] += dp[v][0];
                dp[x][0] += max(dp[v][0], dp[v][1]);
        }
        dp[x][1] += weight[x];
}
dfs(root) // final answer is max(dp[root][0], dp[root][1]);
```

Graph

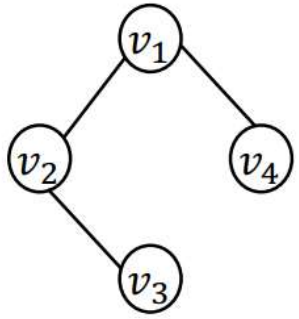
- Linked list and linked table
- **Adjacency list**: Each node is associated with a linked list that stores all neighbors. We map an ID for each node.
  - Space : $O(n+m)$



G



- **Adjacency matrix**: A $n \times n$ two-dimensional matrix 0 where A[u][v] = 1 if u and v are connected.
  - Space : $O(n^2)$



G

	v_1	v_2	v_3	v_4
v_1	0	1	0	1
v_2	1	0	1	0
v_3	0	1	0	0
v_4	1	0	0	0

Comparison1	Adjacency list	Adjacency table
check whether an edge exists	$O\ (\deg(v))$	$O\ (1)$
traverse adjacency edge	$O\ (\deg(v))$	$O\ (n)$
insert an edge	$O\ (1)$	$O\ (1)$
delete an edge	$O\ (\deg(v))$	$O\ (1)$

Shortest Path on Graph

1. **Dijkstra** (based on greedy)

- Single-source, non-negative edge weights
- Iteratively selecting the node with the smallest known distance
- Updating the shortest path estimates for its neighbors
- Repeating this process until all nodes have been visited
- The algorithm ensures that once a node's shortest path is determined, it will not change. **(Edge weights must be non-negative)**

```cpp
void dijkstra(int source){
    for(int i = 1;i <= n;i++) dist[i] = inf;
    dist[source] = 0;
    priority_queue<pair<int,int>,vector<pair<int,int>>,greater<pair<int,int>>> pq;
    pq.push({0,source});
    while(!pq.empty()){
        int temp = pq.top().second;
        pq.pop();
        if(vis[temp]) continue;
        vis[temp] = true;
        for(int v:adj[temp]) if(dist[v] > dist[temp] + edgeweight[temp][v]){
            dist[v] = dist[temp] + edgeweight[temp][v];
            pq.push({dist[v], v});
        }
    }
}
```

Implement by heap (priority queue)

Time complexity:

$O(m \log(n))$

2. Floyd (based on dynamic programming)

- All sources, arbitrary edge weights
- Considering each node as an intermediate point.
- Starting with direct distances between nodes
- Iteratively updates the distance matrix by checking if a path through an intermediate node offers a shorter path.
- This process continues until all possible pairs have been considered.

Initial version of floyd: `dist[k][i][j]`

`dist[k][i][j]` means consider only the induced subgraph of the first k vertices as intermediate part of the path between i and j.

```
• for(int k=1;k<=n;k++)
•     for(int i=1;i<=n;i++) for(int j=1;j<=n;j++)
•         dist[k][i][j]=min(dist[k-1][i][j],
•                           dist[k-1][i][k]+dist[k-1][k][j]);
```

The k can be ignored in the array

```
• for(int k=1;k<=n;k++)
•     for(int i=1;i<=n;i++) for(int j=1;j<=n;j++)
•         dist[i][j]=min(dist[i][j],
•                        dist[i][k]+dist[k][j]);
```

Homework 4 problem 1 solution

1. Use Floyd or Dijkstra to pre-calculate the distance between any two vertices. Then consider brute force over all possible ways to traverse.

There are k edges that you must traverse. generate all $k!$ permutation and enumerate which side to enter every edge, totally 2^k ways.

Time complexity:

$O\ (n^3 + q \times k! \times 2^k)$ if you use Floyd.

$O\ (m \times n \times \log(n) + q \times k! \times 2^k)$ if you use Dijkstra.

Homework 4 problem 2 solution

1. Cosider Maximum Spanning Tree: (Easiest solution)

Sort the edges in descending order, and add edges gradually until source and destination are in one connected component.

Use a disjoint union set to determine whether two vertices have the same father vertex, whose complexity is $\log(n)$

Time complexity:

$$O\left(q \times m \times (\log(m) + \log(n))\right)$$

Homework 4 problem 2 solution

2. Consider binary search. (Xinyan Zhou's idea)

We guess the answer is p, binary search p. Then we check whether p is a valid solution. If we can start from s and end at t if we only use edges whose weights are greater than or equal to p, then it's a valid solution. Final answer is the maximal valid solution because it must correspond to an edge weight

Time complexity: $O(q \times \log(\max weight) \times n)$

where Max_Weight is the maximal weight of all edges

Homework 4 problem 2 solution

3. Consider Widest Path Dijkstra. (Prof. Fang's solution)

- It's the same idea as Shortest Path Problem. The only difference is:
- `dist[v] = min(dist[u] + edgeweight, dist[v]);` (shortest path)
- `dist[v] = max(min(dist[u], edgeweight), dist[v]);` (widest path)
- Can be implemented greedily. (Initially `dist[source] = inf` and others are 0)
- Dijkstra again…
- Time Complexity: $O\ (q \times m \times \log(n))$

Homework 4 problem 3 solution

Some useful observation: If your SP is greater than 100, you can reach anywhere you want, without losing HP.

Why not use `dist[i][j]`, a 2-D array, which means the minimum HP you need if you start from `i` with initial SP = `j`, to arrive at `t`. Dijkstra will help you run the shortest path on this 2-D array. We have totally 10000×100 $(n \times k)$ nodes on this virtual graph we construct, treating each `dist[i][j]` as a node.

Homework 4 problem 3 solution

```
dist[t][0]=0;
priority_queue<pair<int,pair<int,int>>,vector<pair<int,pair<int,int>>>,greater<pair<int,pair<int,int>>>> q;
q.push({0,{t,0}});
while(q.size()){
        int x=q.top().second.first;
        int r=q.top().second.second;
        q.pop();
        if(vis[x][r]||r==100) continue;
        vis[x][r]=1;
        for(int i=head[x];i;i=e[i].next){
                int to=e[i].to;
                int w=e[i].wei;
                if(dist[to][r+1]>dist[x][r]+w/(r+1)){
                        dist[to][r+1]=dist[x][r]+w/(r+1);
                        q.push({dist[to][r+1],{to,r+1}});
                }
        }
}
```

Homework 4 problem 3 solution

```
int ans = inf;
for(int i = 0; i <= 100; i++){
        ans = min(ans, dist[s][i]);
}
for(int i = 1; i <= n; i++){
        ans = min(ans, dist[i][100]);
}
cout << ans << '\n';
```

Sample Final Review

1. [10 marks] State and prove whether the following statements are correct or not:

(1) [5 marks] $n^{1/2} + logn = O(n^{1/2})$

(2) [5 marks] $log_{10}(2^n) = \Theta(n)$

1. (1). True. let $f(n)=n^{\frac{1}{2}}+\log n$.

$$\lim_{n\to\infty}\frac{n^{\frac{1}{2}}+\log n}{n^{\frac{1}{2}}}=1+\lim_{n\to\infty}\frac{\log n}{n^{\frac{1}{2}}}=1.$$

There exists constant $C>1$ s.t. $\lim_{n\to\infty} f(n)\le Cn^{\frac{1}{2}}$. i.e. $f(n)=O(n^{\frac{1}{2}})$

(2) True. $f(n)=\log_{10}2^n=\dfrac{\log_2 2^n}{\log_2 10}=\dfrac{n}{\log_2 10}$.

so there exists constant $C=\dfrac{1}{\log_2 10}$ s.t. $f(n)=Cn$.

$f(n)=O(n)$

Sample Final Review

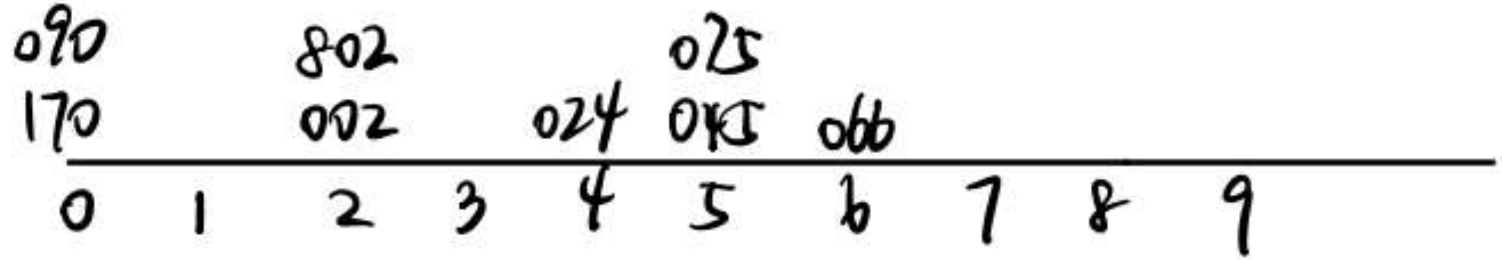
2. [10 marks] Suppose the nodes of a doubly linked list structure are defined as follows:

```
Public class Node {
        public int data;
        public Node next, prev;
};
```

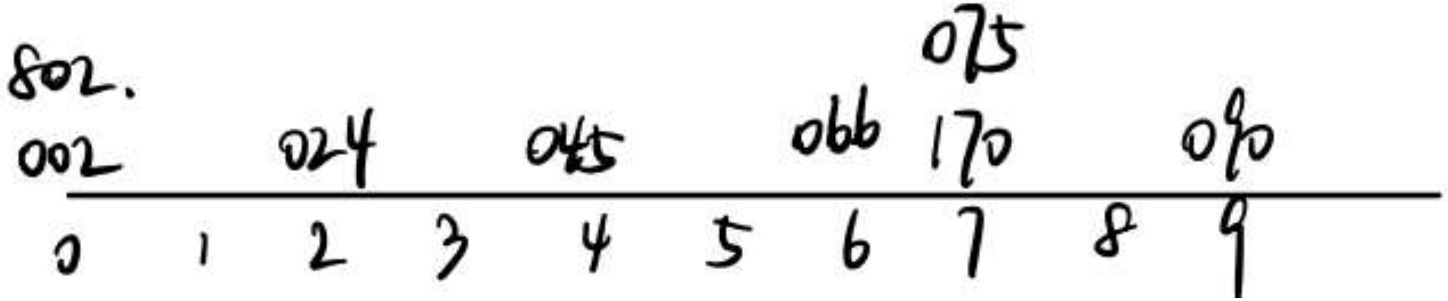
Design an algorithm with pseudocodes which **concatenates** two given lists (the first node of the second list will follow the last node of the first list) and returns the new list. Note that it does not create new nodes; it just rearranges the links of some existing nodes.

```
Node union(Node first, Node second){
	Node ans = first;
	while(first.next != NULL){
		first = first.next;
	}
	first.next = second;
	second.prev = first;
	return ans;
	// we need to return the head of the merged linked list
	// otherwise we will lose the entire linked list.
}
```

① digit 1

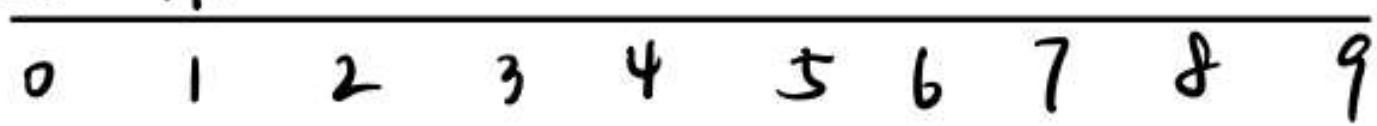


② digit 10. (adjust the elements from left to right



③ digit 100

090
075
066
045
024
002 170 802



from left to right, from bottom to top, we get

002 < 024 < 045 < 066 < 075 < 090 < 170 < 802.

Sample Final Review

4. [10 marks] Given the **inorder sequence** X and **postorder sequence** Y of traversing a binary tree, reconstruct the binary tree such that its **inorder** and **postorder** are X and Y respectively. Here, X and Y are arrays with elements $X[1], X[2], \ldots, X[n]$ and $Y[1], Y[2], \ldots, Y[n]$ respectively, where $n \geq 1$.

(1) [2 marks] Briefly discuss the main idea of reconstruction.

(2) [4 marks] Write the pseudocodes for the reconstruction algorithm.

(3) [4 marks] Analyze the time complexity of the above algorithm using the Θ notation.

```
vector<int> inorder, postorder;

int pos[200001];

Node root = Node(postorder[n-1]);

Node* dfs(int left, int right, int posindex) {

    if (left > right) return nullptr;

    int rootValue = postorder[posindex];

    Node* root = new Node(rootValue);

    int rootPos = pos[rootValue];

    root->right = dfs(rootPos + 1, right, posindex - 1);

    root->left = dfs(left, rootPos - 1, posindex - (right - rootPos) - 1);

    return root;

}

for(int i = 0; i < n; i++){
        pos[inorder[i]] = i;

}

Node* root = dfs(0, n-1, n-1);
```

Sample Final Review

5. [10 marks] Given an array of keys A=[4, 8, 6, 9, 11, 1, 12], use **HeapSort** to sort it:

(1) [5 marks] Draw a sequence of figures to show how a **max-heap** is built on A.

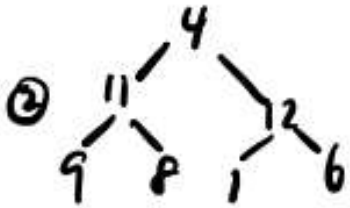
(2) [5 marks] Draw a sequence of figures to show how the elements are sorted **ascendingly** by HeapSort, such that each figure shows an updated max-heap after deleting a key.

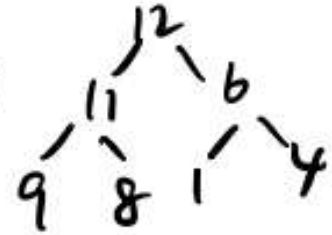
1.



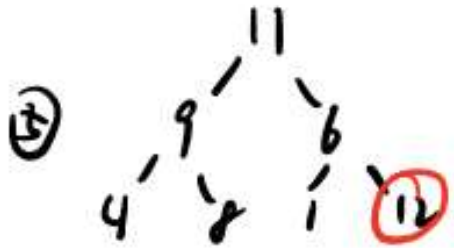
procedure: find the first node that is not leaf. compare it with its children and adjust

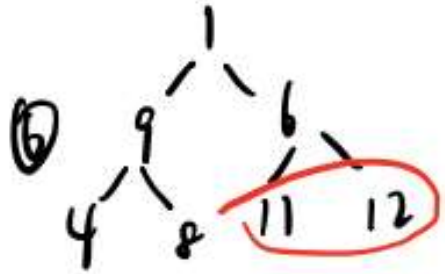
① 

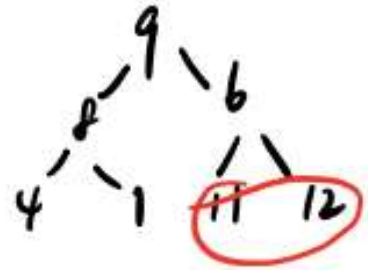
② 

③ 

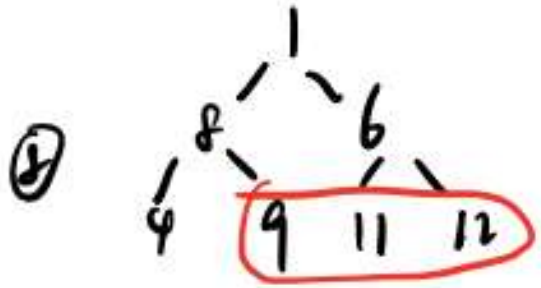
④ 

⑤ 

⑥ 

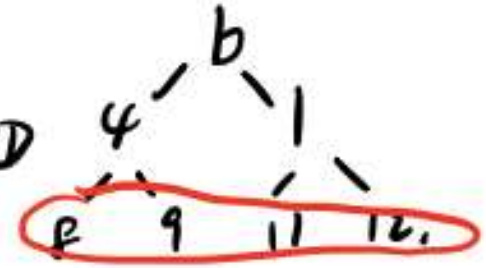
⑦ 

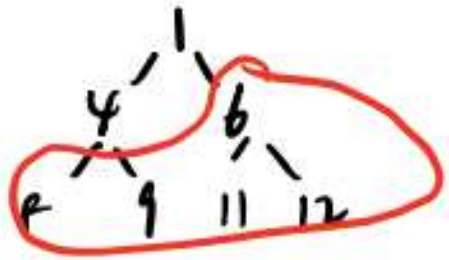
red part is already sorted, dismissed in the later sorting process


8
6
4
9
11
12


8
4
6
1
9
11
12


4
6
8
9
11
12


6
4
1
8
9
11
12


4
6
8
9
11
12


4
1
6
8
9
11
12


1
4
6
8
9
11
12

Sample Final Review

6. [10 marks] Consider a hashtable with separate **chaining** with N buckets and k items.

(1) [2 marks] k/N is the definition of a term used when discussing hashing. What is the name of this term?

(2) [2 marks] Is it necessary that $k < N$?

(3) [3 marks] In the worst-case, how many items could be in a single bucket?

(4) [3 marks] If we resize the table to a table of size $2N$, what is the asymptotic running time in terms of k and N to put all the items in the new table?

6. (1) $\alpha = \frac{k}{N}$ is called load factor.

(2) k is not necessarily smaller than N

(3) k items.

(4) collecting elements : $O(N+k)$ $\alpha' = \frac{k}{2N}$ average insertion $O\left(1+\frac{k}{2N}\right)$

total complexity $= O(N+k+k\cdot\frac{k}{2N}) = O\left(N+k+\frac{k^2}{2N}\right) = O\left(N+\frac{k^2}{2N}\right)$

Sample Final Review

7. [10 marks] Given a binary search tree with root node T, find which value is the **median** value, and delete that value. Assume that for each node p, we can access its left child and right child by *p.leftChild* and *p.rightChild* respectively, and access its key value by *p.key*.

(1) [5 marks] Design an algorithm to solve the above problem and show its main steps.

(2) [5 marks] Use O notation to analyze the time complexity of your algorithm. Your bound should be as tight as possible (as if you are using Θ notation).

```
vector<int> vec;

void dfs(Node t){

    dfs(t.leftChild);

    vec.push_back(t.key);

    dfs(t.rightChild);

}

dfs(T);

int median = vec[(vec.size() - 1) / 2];

Node temp = T;

while(temp.key != median){
        if(temp.key < median){
                temp = temp.rightChild;
        }else{
                temp = temp.leftChild;
        }

}

// delete Node temp and replace it with its successor, ommitted here.
```

Sample Final Review

8. [10 marks] Consider an undirected graph with n nodes and m edges ($m \geq n$), and its adjacent list, where for each node v, its adjacent list is denoted by $Adj(v)$.

(1) [3 marks] What is the time complexity to count the **total number of edges**? Use O notation and the bound should be very tight (as if you are using Θ notation).

(2) [4 marks] Assume that the size of the adjacent list of each node v, denoted by $d(v)$, is known in advance, can we **sort all the nodes according to their degrees** in an ascending order in $O(n)$ time cost? If yes, briefly show the idea; if no, please explain why.

(3) [3 marks] What is the big-O time cost of **removing a node v from the graph**? Notice that removing a node means that all the edges linked to this node will be removed as well, and the bound should be very tight (as if you are using Θ notation).

Answer:

(1) O (n+m) and O (m) are fine

(2) Yes, we can use counting sort or radix sort.

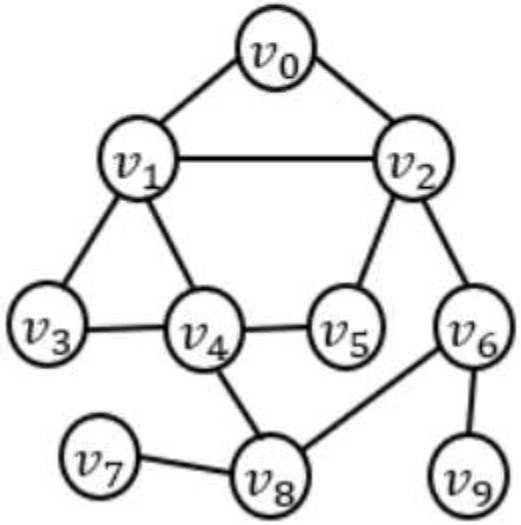
(3) O (n+m) or O(m)

Sample Final Review

9. [10 marks] Consider an undirected graph G_1, which is depicted as below:

(1) [6 marks] Show the sequences of nodes of **BFS traversal** and **DFS traversal** respectively, by assuming that the starting node is set to v_0.

(2) [4 marks] Compare BFS with DFS in terms of time cost and extra space cost, if we use the **adjacent matrix** to represent the graph.



Graph G_1

Answer:

BFS: v0, v1, v2, v3, v4, v5, v6, v8 v9, v7

DFS: v0, v1, v3, v4, v8, v7, v6, v9, v2, v5

BFS: Time: O(V^2) Space: O(V) (implemented by a queue)

DFS: Time: O(V^2) Space: O(V) (use working stack)

Sample Final Review

10. [10 marks] You are given an edge-weighted undirected graph, using the adjacency list representation, together with the list of edges in its minimum spanning tree (MST). Describe an efficient algorithm for updating the MST, when each of the following operations is performed on the graph. Assume that common graph operations (e.g., DFS, BFS, finding a cycle, etc.) are available to you, and don't describe how to re-implement them.

(1) [5 marks] Update the MST when the weight of an edge that **was** part of the MST is **increased**. Show the main ideas of your algorithm and give the order-of-growth running time of your algorithm as a function of V and/or E.

(2) [5 marks] Update the MST when the weight of an edge that **was not** part of the MST is **decreased**. Show the main ideas of your algorithm and give the order-of-growth running time of your algorithm as a function of V and/or E.

(1) Break the edge, use label[i] array to denote which connected component vertex i is in. For the broken edge, DFS or BFS it’s two vertices, to construct the label array. O(V) here. Then sort the edges in ascending order w.r.t weight. O(ElogE) here. Scan the edges and find the minimum weight of an edge which connects two vertices with different labels. O(E) here.

Time Complexity: O(V+ElogE)

(2) Note that there are only V-1 edges in MST

Try to replace every edge in MST with the weight-decreased edge. O(V) here for enumerating. Check whether the graph is connected using DFS/BFS. That is, only start from any single vertex in this graph and run DFS/BFS, check whether every vertex in this graph is visited O(V) for check. If so, then it's a valid replacement. Update the answer.

Time Complexity: O(V^2) (Enumerate * Check)